Calcium in Neuro-VISOR

The model, the solver, the 3D soma and how it is drawn

branch `calcium-model` of `Neuro-VISOR-rosado-calcium`

James Rosado

2 November 2026

Abstract

This note describes the intracellular calcium simulation of the `calcium-model` branch of Neuro-VISOR: the model equations (Borole, Rosado, Neal & Queisser, SIAM J. Appl. Math. 2023), how they are discretized on the neuron graph, the SBDF2 time stepping, and the new parts: the soma resolved as a 3D lattice of compartments (volume diffusion), the ER drawn inside a see-through membrane, the soma colored from the lattice by trilinear interpolation, and the soma screen. Every part is tied to the file and line that implements it, every parameter is listed with its value and the place to change it. The figures come from two sources: stills rendered by Unity itself from the running VR scene (Section 7), and plots made by running the project's own solver code outside Unity on the same cell with the same stimulus (Section 8).

Contents

1 Overview **3**

2 The model **3**
2.1 Species and equations . . . 3
2.2 Parameters . . . 5

3 Discretization on the neuron graph **6**
3.1 Finite volumes on the cable . . . 6
3.2 Choice of refinement . . . 6
3.3 Thin branches: `minRadius` . . . 7

4 The soma as a 3D lattice (volume diffusion) **7**
4.1 Why . . . 7
4.2 Construction . . . 7
4.3 Known artifact: uneven ER release in the soma . . . 8

5 Time stepping **9**
5.1 SBDF2 . . . 9
5.2 Step size, frames and real time . . . 10

6 What you see: coloring **10**
6.1 From the model to the mesh . . . 10
6.2 The soma from the lattice . . . 10
6.3 The soma screen . . . 11

7 In VR **11**

8 Results on the CA1 cell **15**

9 Where to change things **17**

10 Testing and open points **18**

A Reproducing the figures **18**

1 Overview

The calcium simulation replaces the voltage (Hodgkin–Huxley) solver of Neuro-VISOR by a reaction–diffusion model of four species on the 1D neuron graph: cytosolic calcium c_c, ER calcium c_e, and the buffers calbindin b and calreticulin b_e. The user clicks (raycasts) the cell to let calcium in through the plasma membrane; if enough comes in, calcium-induced calcium release (CICR) through the ryanodine receptors of the ER starts a wave that travels along the dendrite. The membrane is drawn see-through and colored by c_c; the ER is drawn inside it and colored by c_e.

The soma is the one place where a 1D cable is a poor picture: it is a ball of about 20 μm diameter. On the cable a soma is one node, and anything that reaches it is instantly mixed into its whole volume. The branch now resolves it as a 3D grid of small compartments (the *soma lattice*), so a wave entering the soma spreads through it as a front, and the ER inside it empties from the outside in.

File (`Assets/Scripts/C2M2/NeuronalDynamics/...`)	Role
`Simulation/CalciumModel.cs`	Parameters (`CalciumParameters`) and the solver (`CalciumModel`): reactions, fluxes, RyR states, SBDF2. No Unity code.
`Simulation/CalciumGeometry.cs`	*New.* Per-node membrane factors, volumes and diffusion operators; `Cable` builds them for the 1D cable. No Unity code.
`Simulation/SomaLattice.cs`	*New.* The soma as a 3D lattice, joined to the cable; the trilinear `Stencil` and `VertexOverrides` used for coloring. No Unity code.
`Simulation/CalciumSolver.cs`	The Unity component: loads the cell, refines it, builds lattice and model, receives clicks and leaks, hands values to the visualization, builds the ER.
`Simulation/NDSimulation.cs`	Base class of the neuron solvers; new hook `Adjust3DValues` for 3D values that do not come from the 1D graph.
`Visualization/ERSurface.cs`	The ER mesh inside the cell and its colors; overrides for the soma.
`Visualization/SomaCamera.cs`	*New.* The soma screen on the wall: a live cross section of the soma.
`Visualization/Mapping/UGXReader.cs`	32-bit mesh indices above 65,535 vertices (fix for large cells).
`Assets/Editor/CalciumCapture.cs`	*New.* Editor script that renders the stills of Section 7 without a headset.

Table 1: Where the calcium simulation lives.

State of the branch. The model, solver, ER and the 32-bit index fix are committed (`97ad53eb`, `beb71123`, `e983050d`; the fixes for the 228-18 cell are written up in `CALCIUM_228-18_FIXES.md`, `624d1a0f`). The soma lattice, `CalciumGeometry`, the soma screen, the trilinear soma coloring and the capture script are **not committed yet**. Of the uncommitted code, the Unity-free files were compiled and run with .NET 8, and Unity compiled all of it and ran it for the stills of Section 7.

2 The model

2.1 Species and equations

On the cable a node is a cylinder of radius R with the ER a coaxial cylinder of radius $r = \rho R$ inside it (ρ = `erRadiusRatio`). The cytosol is the annulus between them (Figure 1). With x the arc

length along the dendrite and $A_c = \pi(R^2 - r^2)$, $A_e = \pi r^2$ the cross sections,

$$\partial_t c_c = \frac{1}{A_c}\partial_x(A_c D_c\,\partial_x c_c) + \underbrace{\kappa_b^-(b_{\mathrm{tot}} - b) - \kappa_b^+ b\,c_c}_{\text{calbindin}} + g_{PM}\,J_{PM} + g_{ERc}\,J_{ERM}, \tag{1}$$

$$\partial_t c_e = \frac{1}{A_e}\partial_x(A_e D_e\,\partial_x c_e) + \underbrace{\kappa_{be}^-(b_{e,\mathrm{tot}} - b_e) - \kappa_{be}^+ b_e\,c_e}_{\text{calreticulin}} - g_{ERe}\,J_{ERM} + g_{ERe}\,J_{SOC}, \tag{2}$$

$$\partial_t b = \frac{1}{A_c}\partial_x(A_c D_b\,\partial_x b) + \kappa_b^-(b_{\mathrm{tot}} - b) - \kappa_b^+ b\,c_c, \tag{3}$$

$$\partial_t b_e = \frac{1}{A_e}\partial_x(A_e D_{be}\,\partial_x b_e) + \kappa_{be}^-(b_{e,\mathrm{tot}} - b_e) - \kappa_{be}^+ b_e\,c_e. \tag{4}$$

The membrane fluxes J are flux densities [mol/(m^2 s)]; the factors g turn them into rates per volume:

$$g_{PM} = \frac{A_{pm}}{V_c} = \frac{2R}{R^2 - r^2}, \qquad g_{ERc} = \frac{A_{er}}{V_c} = \frac{2r}{R^2 - r^2}, \qquad g_{ERe} = \frac{A_{er}}{V_e} = \frac{2}{r}, \tag{5}$$

with A_{pm}, A_{er} the plasma and ER membrane areas and V_c, V_e the cytosol and ER volumes. In the soma lattice the same equations hold per cell, with the areas and volumes of the cell (Section 4).

Plasma membrane. $J_{PM} = -J_P - J_N + J_{l,p} + J_{SYN}\,(+J_{A\beta})$, positive into the cytosol:

$J_P = \rho_P I_P \dfrac{c_c^2}{K_P^2 + c_c^2}$	PMCA pump
$J_N = s_N\,\rho_N I_N \dfrac{c_c}{K_N + c_c}$	NCX, $s_N =$ `ncxScale`
$J_{l,p} = v_{l,p}\,(c_o - c_c)$	PM leak
$J_{SYN} = j_{\mathrm{rls}}\Big(1 - \frac{t - t_0}{\tau_{\mathrm{rls}}}\Big)$ on $[t_0, t_0 + \tau_{\mathrm{rls}}]$ (pulse); j_{rls} while held	click
$J_{A\beta} =$ `amyloidLeakFlux`	Aβ leak sites

ER membrane. $J_{ERM} = J_{RyR} - J_S + J_{l,e}$, positive out of the ER into the cytosol, and the store-operated influx J_{SOC} that refills the ER from outside:

$J_{RyR} = \rho_R\,(o_1 + o_2)\,I_R \dfrac{c_e - c_c}{c_{e,\mathrm{ref}}}$	ryanodine receptor
$J_S = \rho_S I_S \dfrac{c_c}{(K_S + c_c)\,c_e}$	SERCA pump
$J_{l,e} = v_{l,e}\,(c_e - c_c)$	ER leak
$J_{SOC} = \rho_{SOC}\,f(c_e)\,\dfrac{R}{R_{\mathrm{ref}}}\,\dfrac{I_{SOC}\,\ln(10\,c_o/c_e)}{F\,z\,A_{\mathrm{Orai}}}$	SOC (ORAI)
$f(c_e) = \begin{cases} 10 & c_e \le c_{\mathrm{cut}} \\ e^{(c_{e,\mathrm{eq}} - c_e)/1\,\mu\mathrm{M}} - 1 & c_e > c_{\mathrm{cut}} \end{cases}$	SOC open probability

RyR states. Four states with $o_1 + o_2 + c_1 + c_2 = 1$:

$$\dot o_2 = k_b^+ c_c^3\,o_1 - k_b^- o_2, \qquad \dot c_1 = k_a^- o_1 - k_a^+ c_c^4\,c_1, \qquad \dot c_2 = k_c^+ o_1 - k_c^- c_2. \tag{6}$$

The steep c_c^3, c_c^4 dependence is what makes CICR all-or-nothing: below a threshold the RyR stay closed, above it they open, release ER calcium, which opens more RyR.

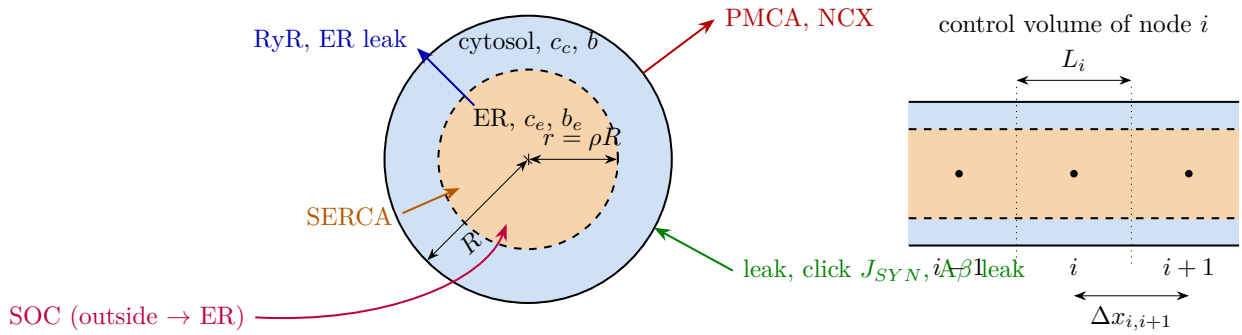


Figure 1: The cable picture. Left: cross section of a dendrite, with the fluxes of Eqs. (1)–(4). Right: the control volume of node i along the graph: half of every edge at the node.

Equilibrium. The leak velocities are not free parameters: they are set so that the resting state ($c_c = 50\,\text{nM}$, $c_e = 250\,\mu\text{M}$, buffers and RyR at their steady states) has zero net flux,

$$v_{l,e} = \frac{J_S - J_{RyR}}{c_{e,\text{eq}} - c_{c,\text{eq}}}, \qquad v_{l,p} = \frac{J_P + J_N}{c_o - c_{c,\text{eq}}} \quad \text{(at rest)}. \tag{7}$$

With the code defaults this gives $v_{l,e} = 4.04 \times 10^{-8}\,\text{m/s}$ and $v_{l,p} = 8.55 \times 10^{-9}\,\text{m/s}$, at rest $b = 37.3\,\mu\text{M}$, $b_e = 3.2\,\text{mM}$, and the RyR mostly in c_1 ($o_1 + o_2 = 3.2 \times 10^{-4}$). They are recomputed whenever `ncxScale` changes, so the rest state stays a steady state.

Not in this model. The voltage-dependent terms of the paper (VDCC flux, Hodgkin–Huxley coupling) and IP_3 receptors.

2.2 Parameters

All values are SI: lengths in m, time in s, concentrations in $\text{mol/m}^3 = \text{mM}$, so $50\,\text{nM} = 50 \times 10^{-6}$. They are the fields of `CalciumParameters` in `Simulation/CalciumModel.cs`, lines 20–146, with the values of the reference MATLAB code (`github.com/NeuroBox3D/CalcSim`) unless marked.

Symbol	Code name	Value	Meaning
Diffusion			
D_c	`Dc`	220×10^{-12} m^2/s	cytosolic calcium
D_b	`Db`	20×10^{-12} m^2/s	calbindin
D_e	`Dce`	10×10^{-12} m^2/s	ER calcium
D_{be}	`Dbe`	27×10^{-12} m^2/s	calreticulin
Buffers			
$\kappa_b^{\pm}$	`kbPos`, `kbNeg`	27×10^3 /(mM s), 19 /s	calbindin binding / unbinding
$\kappa_{be}^{\pm}$	`kbePos`, `kbeNeg`	100 /(mM s), 200 /s	calreticulin binding / unbinding
b_{tot}	`bTot`	$40\,\mu\text{M}$	total calbindin
$b_{e,\text{tot}}$	`beTot`	3.6 mM	total calreticulin
Concentrations and geometry			
$c_{c,\text{eq}}$	`ccEq`	50 nM	resting cytosolic calcium
$c_{e,\text{eq}}$	`ceEq`	$250\,\mu\text{M}$	resting ER calcium
c_o	`co`	1 mM	extracellular calcium
ρ	`erRadiusRatio`	**0.625** (25/40)	ER radius / dendrite radius[†]
Plasma membrane			
ρ_P, I_P, K_P	`rhoP`, `IP`, `KP`	500×10^{12} /m^2, 1.7×10^{-23} mol/s, 60 nM	PMCA
ρ_N, I_N, K_N	`rhoN`, `IN`, `KN`	15×10^{12} /m^2, 2.5×10^{-21} mol/s, 1.8 mM	NCX
s_N	`ncxScale`	**5**	NCX density multiplier[‡]

Symbol	Code name	Value	Meaning
ER membrane			
ρ_S, I_S, K_S	`rhoS`, `IS`, `KS`	2390×10^{12} /m^2, 6.5×10^{-24}, 180 nM	SERCA
ρ_R, I_R	`rhoR`, `IRref`	3×10^{12} /m^2, 3.5×10^{-18} mol/s	RyR density, current
$c_{e,\mathrm{ref}}$	`ceRef`	$250\,\mu$M	ER calcium of the RyR reference current
$k_a^{\pm}$	`kaPos`, `kaNeg`	1500×10^{12} m^{12}/(mol^{4}s), 28.8 /s	RyR rates
$k_b^{\pm}$	`kbRPos`, `kbRNeg`	1500×10^{9} m^9/(mol^{3}s), 385.9 /s	RyR rates
$k_c^{\pm}$	`kcPos`, `kcNeg`	1.75 /s, 0.1 /s	RyR rates
$I_{SOC}, A_{\mathrm{Orai}}$	`ISOC`, `AOrai`	2.1×10^{-15} A, 0.25×10^{-18} m^2	SOC current, channel area
$\rho_{SOC}, R_{\mathrm{ref}}$	`rhoSOC`, `socRefRadius`	10^{-8} /m^2, $0.4\,\mu$m	SOC density scale
c_{cut}	`ceCutoff`	$247.602\,\mu$M	SOC fully open below
F, z	`faraday`, `zCa`	96485 C/mol, 2	
Interaction (Neuro-VISOR)			
j_{rls}	`influxAmplitude`	5×10^{-8} mol/(m^{2}s)	influx density of a click
τ_{rls}	`influxDuration`	10 ms	duration of a pulse
$J_{A\beta}$	`amyloidLeakFlux`	10^{-8} mol/(m^{2}s)	Aβ leak at a clamp[§]
$R_{\min}$	`minRadius`	$0.1\,\mu$m	thinner nodes are raised to it
–	`radiusWeighted-Diffusion`	true	$\frac{1}{A}\partial_x(A\,D\,\partial_x u)$ instead of $D\,\partial_x^2 u$

Table 2: Model parameters (`CalciumParameters`). [†]The paper uses 0.15/0.4 = 0.375, and the comment in the code still says so; the current default is 0.625. A larger ER changes the physics, not only the picture (the factors (5), the stored calcium). [‡]1 in the paper; 5 clears a wave about 4 times faster. [§]The paper's constants give 6×10^{-22}, far below the resting efflux; this default is twice the resting efflux.

3 Discretization on the neuron graph

3.1 Finite volumes on the cable

Each node i owns a control volume: half of each edge at it (Figure 1, right), of length $L_i = \frac{1}{2}\sum_{j\in\mathcal{N}_i}\Delta x_{ij}$. The diffusion term becomes

$$\frac{1}{A}\partial_x(A\,\partial_x u)\Big|_i \approx (Mu)_i = \frac{1}{L_i}\sum_{j\in\mathcal{N}_i}\frac{A_{ij}}{A_i}\,\frac{u_j - u_i}{\Delta x_{ij}}, \qquad A_{ij} = \frac{2A_iA_j}{A_i + A_j}, \tag{8}$$

which handles branch points and unequal edges. With `radiusWeightedDiffusion = false`, $A_{ij}/A_i = 1$, which is the MATLAB code's stencil. All four species share the same operator on the cable, since the ER cross section is a fixed fraction ρ^2 of the cytosolic one.

In general (cable and lattice alike) the geometry is a `CalciumGeometry` (`Simulation/CalciumGeometry.cs`): per node the factors $g_{PM} = A_{pm}/V_c$, $g_{ERc} = A_{er}/V_c$, $g_{ERe} = A_{er}/V_e$, the volumes V_c, V_e, and two sparse operators M_c (for c_c, b) and M_e (for c_e, b_e) with rows $M_{ij} = A_{ij}/(\Delta x_{ij}V_i)$, $M_{ii} = -\sum_j M_{ij}$. The model conserves calcium wherever $V_iM_{ij} = V_jM_{ji}$, i.e. the flux leaving i for j is the one arriving at j. `CalciumGeometry.Cable` (line 48) builds the cable; `BuildCableOperator` (line 103) is Eq. (8).

3.2 Choice of refinement

A CICR front is a few μm wide. On edges much longer than that the wave stalls at every node: on a 64 μm test beam, 2 μm edges propagate in steps, 4 μm at half speed, 8 μm not at all. `RefineForCalcium` (`CalciumSolver.cs`:127) therefore raises the refinement level until every edge is at most `maxEdgeLength` = 1 μm, whatever level was picked in the preview. For the CA1 cell of this note:

refinement	0	1	2	3	4	**5**
1D nodes	3027	3166	3591	4675	7089	**12648**
longest edge [μm]	22.3	10.5	5.04	2.47	1.30	**0.67**

3.3 Thin branches: `minRadius`

The factors (5) scale with $1/R$. The reactions are explicit (Section 5), so on very thin branches the step $\Delta t = 50\,\mu$s becomes unstable once a click raises the calcium there (the 228-18 cell, with radii down to $0.065\,\mu$m, diverged 45 ms after a pulse). Nodes thinner than `minRadius` $= 0.1\,\mu$m are raised to it, for the membrane factors and for the radius-weighted diffusion; `CalciumSolver.PreSolve` logs how many. The CA1 cell of this note has none. Note that this limit was checked at $\rho = 0.375$; at $\rho = 0.625$ the factor g_{PM} at $R = 0.1\,\mu$m is 40% larger (3.3×10^7 against 2.3×10^7 /m), so the check should be redone if thin cells diverge again.

4 The soma as a 3D lattice (volume diffusion)

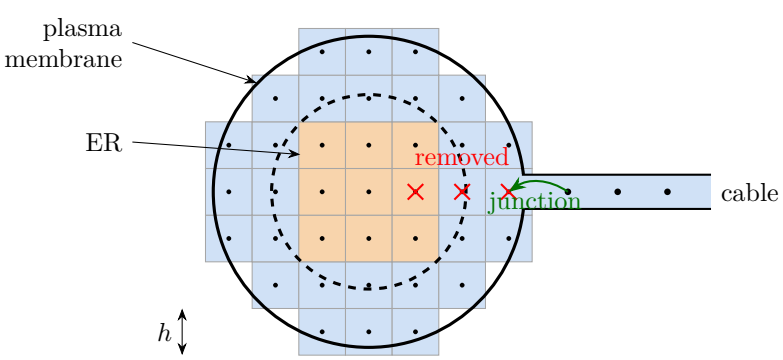
4.1 Why

The soma of the CA1 cell is a single 1D node of radius $10.2\,\mu$m. As one node, calcium entering it from a dendrite is spread over the whole soma at once, and the ER release it triggers happens everywhere in the soma at once. In a real soma calcium needs about $r^2/D \approx 0.5$ s to diffuse from the membrane to the center of a $10\,\mu$m soma, so the signal arrives as a front, and the ER near the membrane releases first. Figure 9 shows how different the two pictures are on the same stimulus.

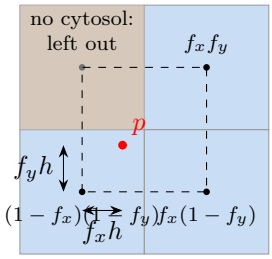
4.2 Construction

`SomaLattice.Build` (`SomaLattice.cs`:71) builds the lattice from the 1D graph, before the solve thread starts (`CalciumSolver.BuildLattice`, line 303):

1. **Sphere.** The soma is a sphere around the mean position of the soma nodes (the `soma` subset of the `.ugx`), with the largest soma node radius R; the ER a concentric sphere of radius ρR. If $R < 3h$ the soma stays one node (`MinCellsPerRadius`).
2. **Cells** (`MakeCells`, line 95). Cubes of side $h =$ `somaCellSize` on a grid centered on the soma; a cube is a cell if its center lies inside the sphere.
3. **Volumes** (`SampleVolumes`, line 135). 6^3 sample points per cell give each cell's cytosol and ER share; the totals are rescaled to the exact sphere and shell volumes.
4. **Membrane areas** (`SampleAreas`, line 174). Points spread evenly over each sphere (Fibonacci lattice, about 20 per cell face), each carrying an equal share of the sphere's area, go to the nearest cell of the right phase: $A_{pm,k}$ (plasma membrane) and $A_{er,k}$ (ER membrane).
5. **Model volumes.** A cell with membrane but only a sliver of volume would get a huge A/V and stiff reactions; its phase volume is floored at `MinMembraneShare` $= 20\%$ of the cube, and a phase the cell does not have at `AbsentShare` $= 5\%$ (it takes no part, but the operator stays regular).
6. **Cell–cell coupling** (line 320). Neighbors through a face exchange through the face's share of each phase, $A_{kj} = h^2 \min(V_k, V_j)/h^3$, so $M_{kj} = A_{kj}/(h\,V_k)$, separately for cytosol (M_c) and ER (M_e). The ER of one cell talks only to the ER of the next, and to its own cytosol through the ER membrane.
7. **Junctions** (line 337). The 1D nodes inside the sphere are taken out of the physics (`MarkRemoved`, line 253). Every 1D edge from an outside node i to a removed node becomes a junction from i to the cell where the edge crosses the soma's surface, through the dendrite's own cross section: $M_{ik} = 1/(L_i\,\Delta x)$ on the cable side and $M_{ki} = A_{c,i}/(\Delta x\,V_k)$ on the cell side (and the same with $A_{e,i}$ for the ER), so $V_i M_{ik} = V_k M_{ki}$ and no calcium is lost.



(a) the lattice and a junction



(b) the stencil (2D view)

Figure 2: (a) A slice through the soma lattice: cells (squares) whose center lies inside the soma; blue cytosol, orange ER; the dendrite's 1D nodes inside the soma (red crosses) are removed and the edge into the soma becomes a junction to the cell where it crosses the surface. (b) The trilinear stencil used to color the soma (Section 6), in 2D: a point gets the weighted values of the cells around it, using only the cells that have its phase, with the weights renormalized.

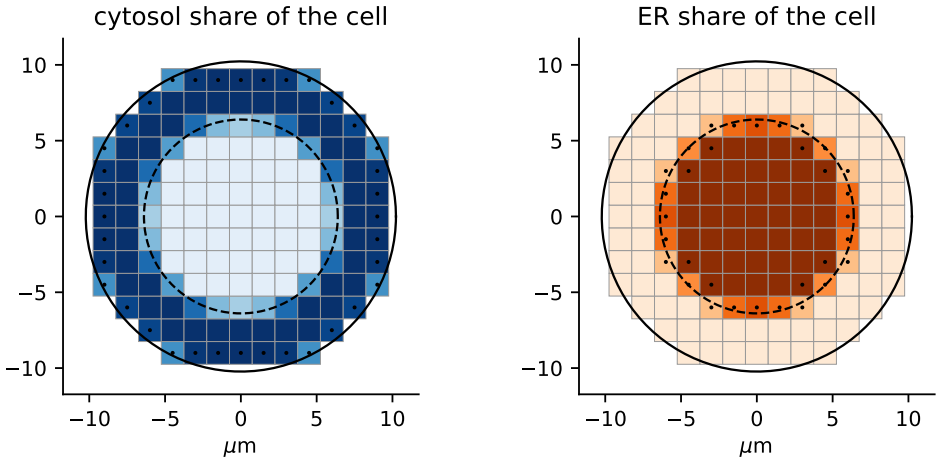


Figure 3: The lattice of the CA1 soma ($R = 10.2\,\mu$m, ER 6.4 μm, $h = 1.5\,\mu$m, 1357 cells): the slice through the center. Shade: share of the cube taken by cytosol (left) and ER (right); dots: cells with plasma (left) or ER (right) membrane area.

8. **Clicks and leaks on the soma** (line 402). A click or clamp at a removed node is redirected to the surface cells: to all of them for a soma node, to those within $\max(3h, 2\,\mu\text{m})$ of where the node lies on the surface otherwise (`CalciumGeometry.InfluxTargets`, applied in `CalciumModel.UpdateInflux`).

The model nodes are the 1D nodes $0, \ldots, N_{1D} - 1$ followed by the cells: cell k is node $N_{1D} + k$. The solver does not know about the lattice; it only sees a `CalciumGeometry` with more nodes and other operators.

4.3 Known artifact: uneven ER release in the soma

The sphere cuts the cubic cells unevenly, so the cells on the ER's surface get very different ER membrane areas for their volumes. On the CA1 soma, over the 308 cells with ER membrane, $g_{ERe} = A_{er}/V_e$ ranges from 0.03 to 3.0 μm^{-1} (median 1.33, coefficient of variation 0.69) and $g_{ERc} = A_{er}/V_c$ from 0.03 to 2.6 μm^{-1}, where a smooth sphere has one value. A cell with a large factor releases and refills its ER faster than its neighbors, so during release the ER surface shows

CA1 cell, refinement 5	
1D nodes	12,648
soma radius, ER radius	10.23 μm, 6.39 μm
lattice cells ($h = 1.5\,\mu$m)	1,357
1D nodes inside the soma (removed)	114 (113 dendrite nodes + the soma node)
junctions (branches joined to the soma)	5
model nodes	14,005
building the lattice / factorizing the 4 operators	38 ms / 214 ms
one step, lattice / soma as one node*	1.98 ms / 1.45 ms

Table 3: The lattice on the CA1 cell (`A-Slide-3-section-2-neuron-1-CA1.vrn`). Unity logs the same numbers: "soma resolved in 3D, 1357 cells of 1.5 um (radius 10.2 um, ER 6.4 um); 113 dendrite nodes inside it replaced, 5 branches joined to it". *Console harness, .NET 8 on the development PC; Unity's timings differ.

round blotches (Figure 7, 160 ms onward). They are in the model values, not in the coloring: the trilinear coloring shows them faithfully, where the nearest-cell coloring turned them into squares. Ways to reduce them: spread each sample point's area over the 8 cells around it with the trilinear weights instead of giving it to the nearest cell (`SphereArea`, `SomaLattice.cs`:181), or compute the factors on a thicker shell of cells, or use a smaller `somaCellSize`.

5 Time stepping

5.1 SBDF2

Write the four species as $X = (c_c, c_e, b, b_e)$, the diffusion as AX (block diagonal: D_cM_c, D_eM_e, D_bM_c, $D_{be}M_e$) and the reactions and membrane fluxes as $F(X)$. The solver uses the second order semi-implicit backward differentiation formula (paper Sec. 2.6), implicit in the stiff, linear diffusion and explicit in the nonlinear reactions:

$$\Big(I - \tfrac{2}{3}\Delta t\, A\Big)X^{n+1} = \tfrac{4}{3}X^n - \tfrac{1}{3}X^{n-1} + \tfrac{4}{3}\Delta t\, F^n - \tfrac{2}{3}\Delta t\, F^{n-1}, \tag{9}$$

started by one IMEX Euler step $(I - \Delta tA)X^1 = X^0 + \Delta tF^0$. Since no two species share a diffusion term, the system splits into four sparse solves of size N. The four matrices $I - \frac{2}{3}\Delta tD_sM_s$ (and the four startup matrices) are LU-factorized once when the model is built (CSparse, `Assets/3rdParty/CSparse`, minimum degree ordering), so a step costs four forward/back substitutions and the reaction loop.

The RyR states (6) are advanced first in each step, by backward Euler at every node; the 3×3 linear system per node is solved by Cramer's rule (`UpdateRyR`, `CalciumModel.cs`:412), as in the MATLAB code.

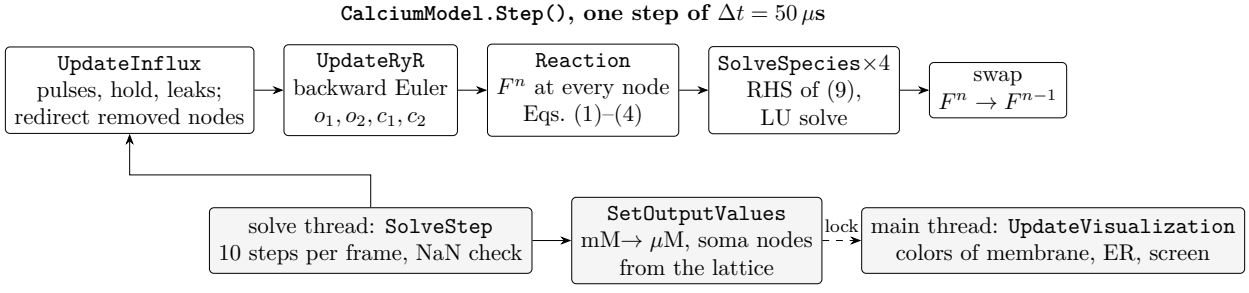


Figure 4: One solver step and one frame.

5.2 Step size, frames and real time

- Δt = `solverTimeStep` = 50 μs. The paper uses 10 μs; 50 μs reproduces its wave peaks within 0.1% and arrival times within about 1% (NMO_150605 neuron and the 64 μm beam).
- `substepsPerFrame` = 10 steps run per simulation frame, i.e. 0.5 ms of simulated time. A frame lasts at least 10 ms of real time, so simulated time runs at about 1/20 of real time, slower if the steps take longer than the frame.
- After every frame `SolveStep` checks for NaN or infinity; if the solver diverged it logs "Calcium solver diverged at t = ..." and resets to equilibrium.
- The solver state (both SBDF2 levels, F^{n-1}, the RyR states, the step count) is saved and restored with the scene (`GetState`/`SetState`).

Explicit reactions limit Δt: the fastest reaction rates grow with the membrane factors (5), i.e. with $1/R$, and with the RyR's c_c^3, c_c^4 during a wave. That is why thin branches need `minRadius`, and why `ncxScale` = 10 together with a tenfold PMCA diverges at 50 μs.

6 What you see: coloring

6.1 From the model to the mesh

The model works in mM; `SetOutputValues` (`CalciumSolver.cs`:243) hands c_c and c_e in μM to the visualization. The surface mesh vertices are colored by interpolating the 1D values along the edge each vertex belongs to, $u_{3D} = u_{v_1} + \lambda(u_{v_2} - u_{v_1})$ (`NDSimulation.cs`:272). The plasma membrane uses a log color scale from 0.05 to 50 μM (`colorScaleMin/Max`, `logColorScale`), so rest, small rises and waves all show; the ER a linear one from 0 to 250 μM (`erColorScaleMin/Max`), so a full ER is at the top of the scale and it moves down as it empties. Both use the gradient chosen on the control panel.

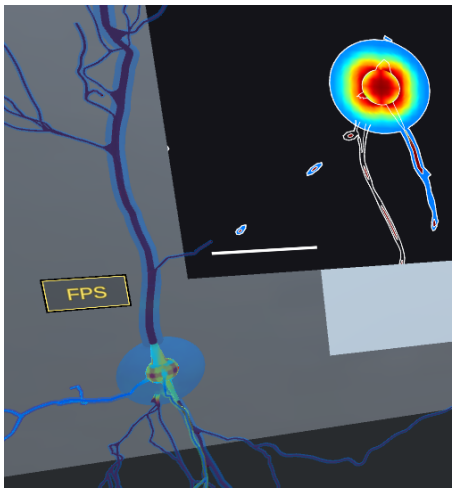
The ER mesh (`ERSurface.Build`, `ERSurface.cs`:56) is a copy of the cell's surface mesh with every vertex pulled toward its point on the centerline to ρ of its distance, so it has the model's radius $r = \rho R$. The membrane is drawn see-through (opacity `membraneOpacity` = 0.4) so the ER shows through it.

6.2 The soma from the lattice

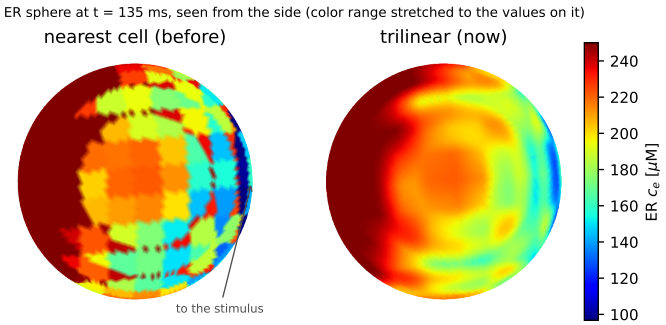
On the soma the 1D values mean nothing any more (its nodes are removed), so the soma's vertices are colored from the lattice. `CalciumSolver.SomaOverrides` (line 347) finds, once, every vertex (of the membrane and of the ER) that is interpolated from a removed 1D node and gives it a stencil (`SomaLattice.Stencil`, line 224, and Figure 2b):

$$u(p) = \sum_{k \in \mathcal{S}(p)} w_k \, u_k, \qquad w_k = \frac{\prod_{a \in \{x,y,z\}} (\delta_{k,a} f_a + (1 - \delta_{k,a})(1 - f_a))}{\sum_{k' \in \mathcal{S}(p)} (\cdots)}, \tag{10}$$

where $\mathcal{S}(p)$ are those of the 8 cells around p that hold the vertex's phase (cytosol for the membrane, ER for the ER), f_a the fractional position of p between the cell centers, and $\delta_{k,a} \in \{0, 1\}$ whether cell k is the upper neighbor along a. If none of the 8 holds the phase, the nearest cell that does is used. Where a branch leaves the soma, the vertex blends this value with the 1D node outside the soma by its λ, so the colors go over smoothly into the dendrite. The stencils are stored in a `VertexOverrides` (`SomaLattice.cs`:458) and applied every frame: for the membrane in `Adjust3DValues` (`CalciumSolver.cs`:375, called by `NDSimulation.cs`:274 after the 1D interpolation), for the ER in `ERSurface.SetValues`.



(a) in VR, nearest cell



(b) same model values, nearest cell vs. trilinear

Figure 5: The checkerboard ER. (a) A screenshot from VR taken before the change: the ER sphere inside the soma shows patches. (b) The ER sphere of the CA1 soma at the moment its surface varies most, seen from the side, colored from the same lattice values both ways.

Before: nearest cell. The first version gave every soma vertex the value of the nearest cell. The values are then constant over each 1.5 μm cell and jump at the cell borders. On the membrane this hardly showed, but the ER sphere spans only a few cells, which differ strongly during release: it looked like a checkerboard (Figure 5). With the stencil, on a test field varying linearly across a soma of radius 5 μm ($h = 1.5\,\mu$m, $\rho = 0.375$), the error of the ER colors fell from 0.45 to 0.11 (rms) and from 1.11 to 0.38 (max), in units of the field's change per μm; on the membrane from 0.56 to 0.40 (rms), less because cells outside the sphere are missing around it.

6.3 The soma screen

`SomaCamera` (`Visualization/SomaCamera.cs`) shows a live cross section of the soma on a wall screen, so a wave arriving can be watched without walking into the cell. The section plane goes through the soma's center and contains the directions in which the nearby branches spread most. The plasma membrane is the outer circle, the ER the inner one; inside the soma's disc every pixel is interpolated from the lattice with the same stencil as above (since this change the screen and the 3D soma share `SomaLattice.Stencil`, so they show the same values). Settings on the `SomaCamera` component: `mode` (Section or Slab), `viewRadiusMicrons`, `sliceOffsetMicrons`, `framesPerSecond`, `resolution`, `screenWidth`. A scene object named `SomaScreenAnchor` fixes the screen's place.

7 In VR

The stills in this section are rendered by Unity from the running scene (`MainScene`), by the editor script `Assets/Editor/CalciumCapture.cs`:

```
Unity.exe -projectPath D:\other_branches\Neuro-VISOR-rosado-calcium
        -executeMethod CalciumCapture.Run -logFile capture.log
```

It loads the CA1 cell through the normal loader at refinement 0 (raised to 5 by the solver), switches the influx to *Hold*, and holds a click for 20 ms of simulated time (from $t = 5$ to 25 ms) on the 7 nodes within 2 μm of the thickest node 35–45 μm from the soma center, a node of the apical trunk ($R = 1.96\,\mu$m, 25.6 μm from the soma along the dendrite). At the times shown it pauses the solver and renders the scene from its own cameras. All parameters are the code defaults of Table 2.

A single click (pulse) on this thick trunk does not start a wave: the influx stays under the CICR threshold (the membrane factors are small on a thick branch). Held for 20 ms it does.

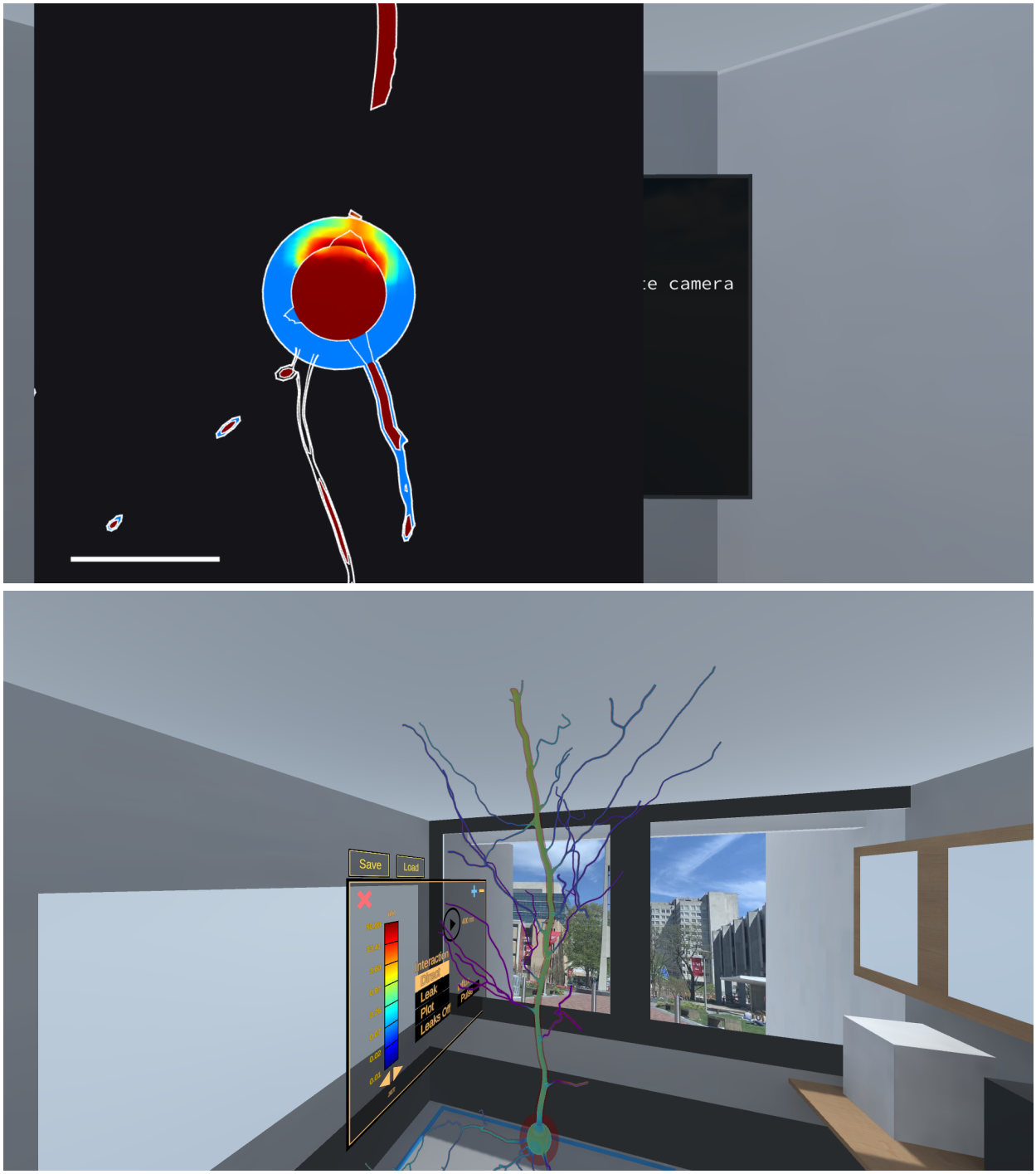
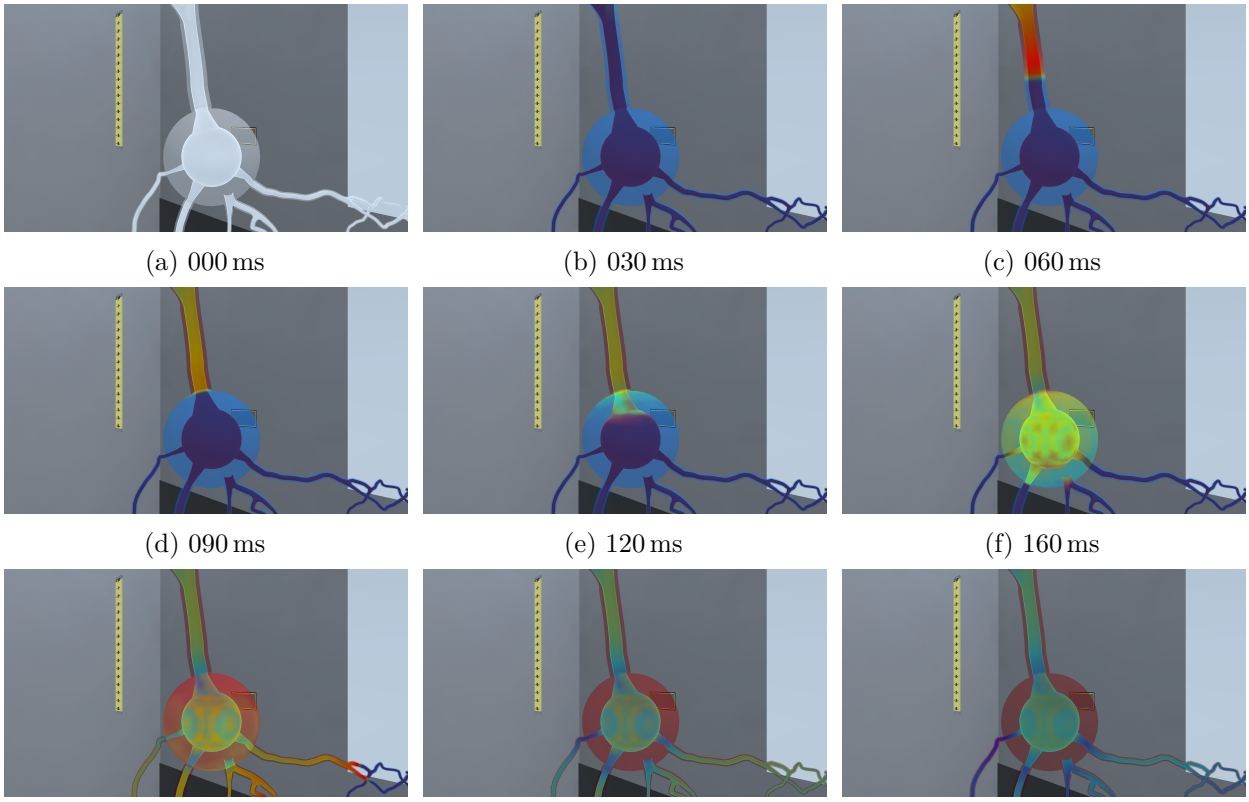


Figure 6: Top: the room at $t = 120\,\text{ms}$, as the desktop camera sees it, with the soma screen. Bottom: the whole CA1 cell at $t = 400\,\text{ms}$.

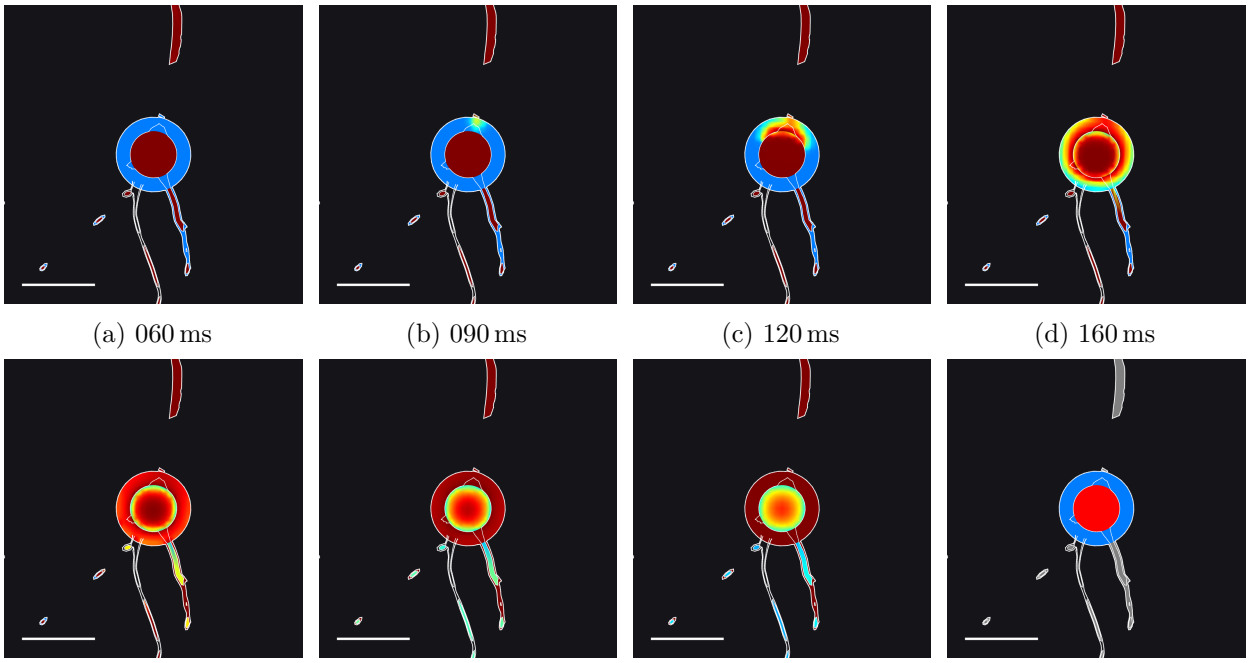


(a) 000 ms (b) 030 ms (c) 060 ms

(d) 090 ms (e) 120 ms (f) 160 ms

(g) 220 ms (h) 300 ms (i) 400 ms

Figure 7: The soma and the stimulated apical trunk in VR. The membrane is see-through and colored by c_c (log, 0.05–50 μM); the ER inside it by c_e (linear, 0–250 μM). The click is held on the trunk from 5 to 25 ms; the wave runs down the trunk, enters the soma at about 80 ms and spreads over it. (The panel's influx toggle still reads "Pulse": the capture script sets the mode directly.) The ER inside the soma is not uniform: see Section 4.3.



(a) 060 ms (b) 090 ms (c) 120 ms (d) 160 ms

(e) 220 ms (f) 300 ms (g) 400 ms (h) 000 ms

Figure 8: The soma screen at the same times (the last panel is the resting state). The outer circle is the plasma membrane, the inner the ER; the cytosol is colored by c_c, the ER by c_e, both interpolated from the lattice. The front enters where the trunk joins the soma and runs around the ER; the ER empties from its surface inward.

8 Results on the CA1 cell

The plots of this section come from the console harness (`scripts/harness`): it compiles `SomaLattice.cs`, `CalciumGeometry.cs`, `CalciumModel.cs` and CSparse unchanged with .NET 8 and runs them on the 1D graph of the CA1 cell exported from the `.vrn` (`scripts/export_graph.py`, refinement chosen as `RefineForCalcium` does), with the stimulus of Section 7. It runs the cell twice, side by side: with the soma lattice (the default, `somaLattice = true`) and with the soma as one node (`somaLattice = false`).

	soma lattice			soma as one node	
t [ms]	mean c_c [μM]	mean c_e [μM]	lowest c_e [μM]	c_c [μM]	c_e [μM]
80	0.07	250	250	0.05	250
100	0.20	250	177	5.2	235
120	1.1	246	115	38	211
160	7.7	220	75	93	181
200	13.8	201	69	115	165
300	38.2	171	74	129	150
400	54.4	153	80	127	144

Table 4: The soma with and without the lattice, same cell and stimulus.

What the lattice changes:

- **Arrival.** As one node the soma's calcium rises everywhere at once to 38 μM by 120 ms. With the lattice, the rise starts at the junction with the trunk and travels around the ER (Figure 11); at 120 ms the soma mean is only 1.1 μM.
- **ER release.** The ER cells next to the front release first: at 120 ms the lowest ER cell is at 115 μM while the soma's ER as a whole is still at 246 μM. The ER empties from its surface inward.
- **Peak.** As one node the soma saturates at about 130 μM within 200 ms; with the lattice the soma fills over hundreds of ms and its mean is 54 μM at 400 ms.
- **Cost.** The lattice adds 1357 nodes to 12,648 and about 37% to the step time (Table 3).

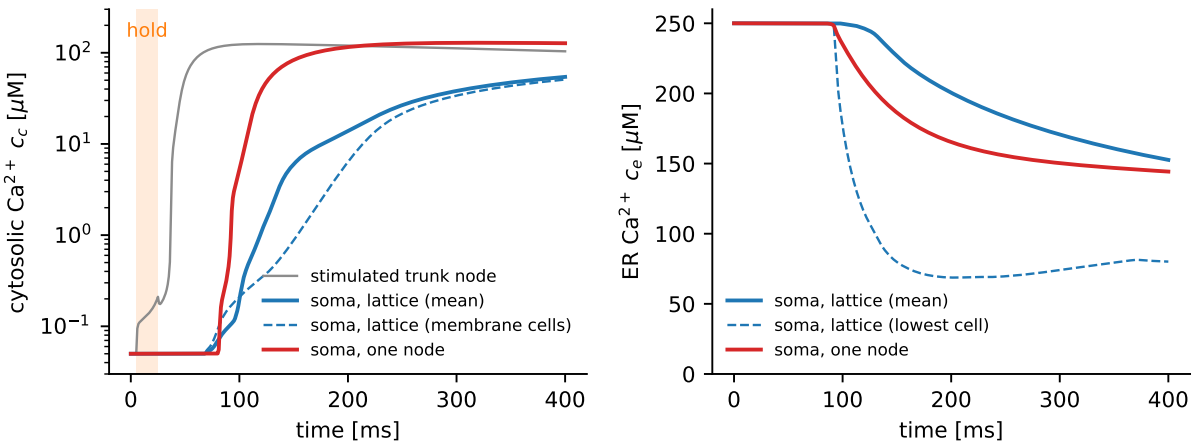


Figure 9: Left: cytosolic calcium at the stimulated trunk node and in the soma. Right: ER calcium in the soma. Blue: soma lattice (solid: volume-weighted mean; dashed: the membrane cells, or the cell with the lowest ER calcium); red: the soma as one node. The shaded band is the 20 ms hold.

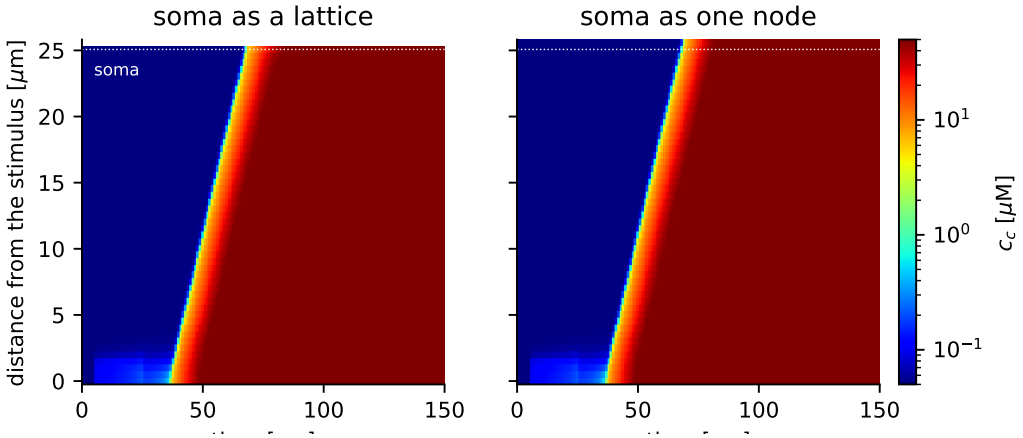


Figure 10: Cytosolic calcium along the path from the stimulus to the soma (25.6 μm) over time. The calcium let in during the hold sets off release at the stimulus, above 10 μM at 40 ms; the wave reaches the soma's edge at 72 ms (about 0.8 μm/ms). Inside the soma (white dotted line) the lattice replaces the 1D nodes, so the left panel is blank there. With the soma as one node (right), the nodes inside the soma rise to about 125 μM and stay there.

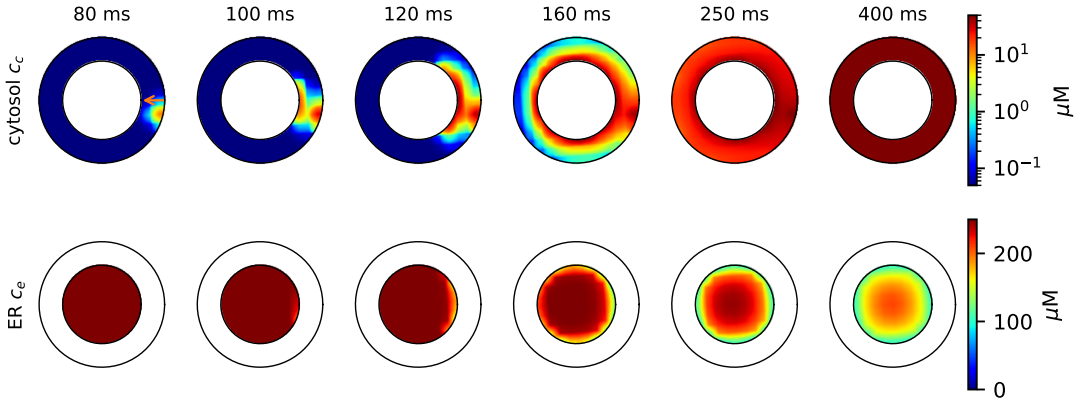


Figure 11: Sections through the soma (plane through the center and the stimulus, which lies to the right), drawn as the soma screen draws them. Top: cytosol (log scale); bottom: ER (linear). Black circles: plasma and ER membrane.

9 Where to change things

Important: the Inspector does not reach a new cell. `NDSimulationLoader.Load` adds the `CalciumSolver` to a new GameObject with `AddComponent` when a cell is loaded, so every cell starts with the code defaults; a value typed into an Inspector before pressing Play is not used. To change a default, change the code. In the Inspector of a *running* solver, the reaction constants of `CalciumParameters` take effect at the next step (the model reads them every step), but only `ncxScale` also recomputes the leak velocities: changing any other pump or channel constant moves the resting state. Everything used to build the model (radii, ρ, `minRadius`, the diffusion coefficients, Δt, the lattice) is fixed once the cell is loaded.

What	Where	Notes
Biology (Table 2)	`CalciumModel.cs`:20–146	any `CalciumParameters` field
ER radius ratio ρ	`CalciumModel.cs`:51	changes the physics (factors (5), ER volume), the lattice's ER sphere, the ER mesh and the screen. With small ρ the soma's ER spans few cells: lower `somaCellSize` too.
How fast a wave clears	`ncxScale`, line 72	1 (paper) to 5 (default); can change while running
Strength of a click	`influxAmplitude`, line 128; `influxDuration`, line 130	a pulse is below threshold on thick branches; use Hold
Pulse or hold	`InfluxModeToggle` in the scene; `NDSimulationManager.Influx`	Hold: influx while the ray is held
Amyloid-β leaks	`amyloidLeakFlux`, line 138; `AmyloidLeakToggle`	clamps become leak sites
Time step	`solverTimeStep`, `CalciumSolver.cs`:33	explicit reactions: smaller if thin cells diverge
Speed of simulated time	`substepsPerFrame`, line 38	more steps per frame: faster, more CPU per frame
Refinement	`maxEdgeLength`, line 53	larger: coarser and faster, waves may stall
Thin branches	`minRadius`, `CalciumModel.cs`:118	0 turns the floor off
Soma lattice on/off	`somaLattice`, `CalciumSolver.cs`:69	off: the soma is one node
Soma lattice cell size	`somaCellSize`, line 74	1.5 μm: ∼1400 cells for a 10 μm radius; 1 μm: sharper fronts, about twice the cost

What	Where	Notes
Lattice internals	`SomaLattice.cs`:56–60	`MinCellsPerRadius`, `MinMembraneShare`, `AbsentShare`
Colors, cytosol	`colorScaleMin/Max`, `logColorScale`, lines 43–47	also the min/max markers on the color scale
Colors, ER	`erColorScaleMin/Max`, line 89	linear
See-through membrane	`membraneOpacity`, line 84; `showER`, line 57	
Soma screen	`showSomaCamera`, line 63; fields of `SomaCamera`	mode, plane, size, frame rate

Table 5: Where to change things. Lines are in `Assets/Scripts/C2M2/NeuronalDynamics/Simulation/`.

10 Testing and open points

Tested.

- The Unity-free code (lattice, geometry, model, CSparse) compiles and runs with .NET 8; the runs of Section 8 are stable at $\Delta t = 50\,\mu$s for 400 ms on the CA1 cell, with and without lattice.
- Unity 2019.4.40f1 compiles the whole branch and runs it in the scene (Section 7); the lattice it builds is the harness's (1357 cells, 5 junctions).
- The stencil weights sum to 1; the override blend at the soma's edge gives the expected values.

Open.

- **Commit** the lattice, geometry, soma screen, coloring and capture script.
- $\rho = 0.625$: the comment at `CalciumModel.cs`:50 still says 0.375. Check `minRadius` again at this ratio on the thin 228-18 cell.
- **Refinement at runtime**: the `RefinementLevel` setter reloads only the 1D grid; the surface mesh, the map, the lattice and the model stay at the old level (see `CALCIUM_228-18_FIXES.md`).
- **Mass in sliver cells**: the volume floors (`MinMembraneShare`, `AbsentShare`) add a little volume that the geometry does not have; the operators still conserve $\sum Vu$ with the floored volumes.
- **Uneven ER release in the soma** from the cell-to-cell spread of the membrane factors (Section 4.3).
- **A click on a thick branch** is below the CICR threshold; Hold or a larger `influxAmplitude` starts a wave there.

A Reproducing the figures

```
# 1D graph of the cell at the solver's refinement
python3 scripts/export_graph.py <Geometries>/A-Slide-3-section-2-neuron-1-CA1.vrn ca1.txt
# solver run: graph, output dir, end time [s], distance band [um], target radius [um] (99:
    thickest), hold [s]
dotnet run -c Release --project scripts/harness -- ca1.txt out 0.4 35 45 99 0.02
# plots into figures/sim
python3 scripts/make_figures.py out figures/sim
# VR stills into figures/vr (Unity must not have the project open)
Unity.exe -projectPath <project> -executeMethod CalciumCapture.Run -logFile capture.log
pdflatex calcium_writeup.tex # twice, for the contents and references
```

The harness project refers to the source files in the Unity project by absolute path (under `D:/other_branches/Neuro-VISOR-rosado-calcium/Assets`); change them in `scripts/harness/harness.csproj` if the project moves.

Reference

D. Borole, J. Rosado, M. Neal, G. Queisser, *Neuronal resilience and calcium signaling pathways in the context of synapse loss and calcium leaks*, SIAM J. Appl. Math. 83(6), 2023. Reference MATLAB code: `github.com/NeuroBox3D/CalcSim`.